Week 4 Task: Enhancing Web Page Performance and Accessibility

Frontend Web Development Internship

Objective

The objective of this task was to optimize a static webpage for faster loading, cleaner code, semantic HTML and improved accessibility. The final webpage demonstrates practical techniques that help both general users and people who use assistive technologies.

1. Initial Analysis

Common issues considered during the analysis included unnecessary HTML structure, repeated CSS declarations, weak heading hierarchy, missing keyboard focus indicators, poor form labeling, and navigation that could be difficult for screen-reader or keyboard users. The revised version addresses these areas with a simpler document structure and reusable CSS rules.

2. Performance Improvements

Technique	Implementation
Lightweight HTML	Removed unnecessary containers and used semantic elements such as header, nav, main, section, article
Optimized CSS	Used CSS custom properties and reusable classes to reduce repeated declarations.
Efficient selectors	Used simple class, element and state selectors instead of deeply nested selectors.
No external assets	The demonstration page avoids unnecessary images, external fonts and third-party scripts, reducing netwo
Responsive CSS	Used a small set of media queries so the layout adapts without separate mobile pages.
Reduced motion	Added prefers-reduced-motion support for users who prefer less animation.
Metadata	Added viewport and description metadata to support responsive rendering and page understanding.

3. Accessibility Improvements

• **Semantic HTML:** Used header, nav, main, section, article and footer to provide meaningful structure.

• **Skip link:** A visible-on-focus skip link lets keyboard users move directly to the main content.

• **Keyboard navigation:** Interactive elements have clear :focus-visible outlines.

• **Accessible navigation:** The navigation has an aria-label to identify its purpose to assistive technologies.

• **Form labels:** Every input and textarea has an associated label using matching for/id attributes.

• **Heading hierarchy:** The page follows a logical h1, h2 and h3 structure.

• **Contrast:** Text and interactive elements use high-contrast combinations designed to remain readable.

• **Reduced motion:** The page respects the prefers-reduced-motion user preference.

• **Responsive layout:** Content remains usable on smaller screens without horizontal layout dependence.

4. Testing Tools and Techniques

The page can be tested with Google Lighthouse for Performance, Accessibility, Best Practices and SEO. WebPageTest can be used to examine loading behavior, requests and rendering. WAVE and axe DevTools can be used to identify accessibility issues such as missing labels, structural problems and ARIA misuse. Browser keyboard testing should also be performed by navigating the page using Tab, Shift+Tab and Enter without a mouse.

5. Expected Results

The optimized page should require fewer resources than a page containing unnecessary scripts, images and external dependencies. Its semantic structure should also make the content easier for screen readers and keyboard users to understand and navigate. Actual Lighthouse or WebPageTest scores depend on the browser, device, network and test environment, so scores should be recorded after running the tools rather than being invented in advance.

6. Learning Outcomes

Through this task, I learned that web development is not only about visual design. Performance affects how quickly users can access content, while accessibility ensures that people with different abilities can use the same website. I learned how semantic HTML, efficient CSS, keyboard support, focus states, form labels, responsive design and accessibility testing contribute to a better web experience.

7. Conclusion

The completed webpage demonstrates a practical approach to improving both performance and accessibility. The HTML is structured semantically, the CSS is organized with reusable rules, and accessibility features have been included from the beginning. The project can be further improved by compressing production assets, enabling suitable caching, testing with real assistive technologies, and recording Lighthouse/WAVE/axe results from the final deployment.